

LED 补光网关控制指令与部署手册

适用系统： Linux 工控机 + CH340 USB 转 RS485 + 4 台双通道 LED 驱动器
适用程序： led_linux_gateway/server.py、light_modbus.py（与当前实机联调版本对应）
对接依据： 《LED 补光对接说明》第 5 部分（LED 网关 API）
版本： V1.0 **编写日期：** 2026-10-09

先读提示： 文中的写入/广播命令会直接改变现场灯光。执行前确认现场允许；尤其广播“全灭/全亮”将同时影响四台驱动器。广播接口的 ok:true 仅代表发送成功，需要随后调用 read-all 确认。

1. 系统结构与当前联调状态

通信路径：植物管理平台 → HTTP → Linux 工控机（8090） → USB/RS485 → LED 驱动器。工控机自身作为 LED 网关，无须另设网关设备。

现场工程目录：/home/hzauiot/project/led_linux_gateway。已识别的串口：/dev/ttyUSB0（CH340 / ch341 驱动）；通信参数为 **9600、8N1**。当前用户采用 **Conda base** 环境运行 Python。

已有真实测试结果（来自联调记录）

GET /api/v1/led/health 返回 {"ok":true,"serialConnected":false}：HTTP 服务响应正常；该调用前串口可能尚未打开。

GET /api/v1/led/read?busAddress=0x9A 返回 {"busAddress":"0x9A","ch1":20,"ch2":20}。

GET /api/v1/led/read-all 返回四个设备的 ok:true，当时八个通道光强均为 20。

POST /api/v1/led/set 写 0x9A 的 ch1=30,ch2=20，返回 ok:true，说明收到写命令的确认；**这次写入后的回读值没有在提供的记录中出现**，不能把它写成已完成回读验证。

广播接口的命令已提供，但没有附带独立的广播执行后 read-all 结果，因此需按第 5.8 节自行检查。

2. 设备地址与光谱对应

驱动器地址	CH1 光谱	CH2 光谱	说明
0x9A	730nm	Full Spectrum（全光谱）	两路独立调节
0x9B	630nm	430nm	两路独立调节
0x9C	450nm	530nm	两路独立调节
0x9D	660nm	395nm	两路独立调节

共 **4 台 × 2 通道 = 8 路光强**。协议光强取值为整数 **0~255**：0 表示关闭，255 表示最大协议设置值；这些数值不等于实测光照功率或照度百分比。

注意： 当前代码只接受字符串形式的十六进制设备地址（例如 "0x9A"），**不接受十进制 154**；虽然原对接说明允许双方约定十进制形式，但本工程实际选择了十六进制格式。

3. 基本约定与命令环境

3.1 HTTP 基地址

本机调试（同一台工控机上执行 curl）：

http://127.0.0.1:8090/api/v1

局域网联调（平台后端或另一台电脑访问）：

http://<工控机实际 IP>:8090/api/v1

平台接口 /plant/api/led/... 属于**平台后端接口①**；本手册的 /api/v1/led/... 属于**工控机 LED 网关接口②**，不要混淆。网关 API **不需要** rackKey，该字段由平台侧维护。

3.2 请求与权限

GET：通过 URL 查询参数获取数据。

POST：使用 JSON 正文，并带请求头 Content-Type: application/json。

ch1、ch2 必须同时提供且是 **JSON 整数** 0~255，不能写成字符串 "20" 或小数。

若设置了 LED_API_KEY，**所有五个 API 都必须带** -H "X-API-Key: 你的密钥"。

本机调试时可不设置密钥；但如果监听 0.0.0.0 供外部访问，当前 server.py **强制要求**配置非占位符的 API Key。

3.3 终端快捷变量（本机无密钥调试）

```
BASE="http://127.0.0.1:8090/api/v1"

正式对接或启用 API Key 后，先设置变量（不要把真正密钥写进公开文档、代码仓库或共享截图）：

BASE="http://<工控机实际 IP>:8090/api/v1"
read -rs -p "输入 LED API Key: " LED_KEY; echo
# 调用时附加：-H "X-API-Key: $LED_KEY"
```

下面第 5 节给出 **本机、当前无密钥调试模式** 的可直接复制命令；平台访问时需替换 IP，并附加 API Key。

4. 网关接口速查表

功能	HTTP 方法	路径	是否修改灯光
服务状态检查	GET	/api/v1/led/health	否
读取指定设备	GET	/api/v1/led/read?busAddress=0x9A	否
读取所有设备	GET	/api/v1/led/read-all	否
设置指定设备两通道	POST	/api/v1/led/set	是
广播设置所有设备两通道	POST	/api/v1/led/broadcast/set	是：全部四台

5. 常用操作指令（可直接在工控机终端执行）

5.1 查询网关健康状态

```
curl -i "http://127.0.0.1:8090/api/v1/led/health"
```

响应示例：

```
{"ok":true,"serialConnected":false}
```

ok:true 表示网关进程可访问；serialConnected 仅表示**串口当前是否已经打开**，不代表四台驱动器都在线。首次读取之前显示 false 可能正常。

5.2 读取一台设备的 CH1 和 CH2

示例：读取 0x9A：

```
curl -i "http://127.0.0.1:8090/api/v1/led/read?busAddress=0x9A"
```

读取其他设备只替换 busAddress：

```
curl "http://127.0.0.1:8090/api/v1/led/read?busAddress=0x9B"
curl "http://127.0.0.1:8090/api/v1/led/read?busAddress=0x9C"
curl "http://127.0.0.1:8090/api/v1/led/read?busAddress=0x9D"
```

响应示例（0x9A 的一次真实联调读数）：

```
{"busAddress":"0x9A","ch1":20,"ch2":20}
```

5.3 一次读取全部四台设备（推荐用于核实状态）

```
curl -i "http://127.0.0.1:8090/api/v1/led/read-all"
```

一次真实联调记录为：

```
{
  "devices": [
    {"busAddress": "0x9A", "ch1": 20, "ch2": 20, "ok": true},
    {"busAddress": "0x9B", "ch1": 20, "ch2": 20, "ok": true},
    {"busAddress": "0x9C", "ch1": 20, "ch2": 20, "ok": true},
    {"busAddress": "0x9D", "ch1": 20, "ch2": 20, "ok": true}
  ]
}
```

如果某台异常，仍会返回其他正常设备，例如：

```
{"busAddress": "0x9D", "ch1": null, "ch2": null, "ok": false, "error": "RS485 等待响应超时..."}
```

此时不能把 null 当成 0；ok:false 表示该次读取失败，不表示灯必然已关闭。

5.4 设置指定设备的两个通道

例如将 0x9A 设置为 **CH1=30, CH2=20**：

```
curl -i -X POST "http://127.0.0.1:8090/api/v1/led/set" \
-H "Content-Type: application/json" \
-d '{"busAddress": "0x9A", "ch1": 30, "ch2": 20}'
```

已取得的真实写命令响应：

```
{"ok": true, "busAddress": "0x9A", "ch1": 30, "ch2": 20}
```

ok:true 说明网关收到设备的 Modbus 写入确认帧；建议紧接着回读：

```
curl "http://127.0.0.1:8090/api/v1/led/read?busAddress=0x9A"
```

若现场需要恢复为之前的 20、20：

```
curl -X POST "http://127.0.0.1:8090/api/v1/led/set" \
-H "Content-Type: application/json" \
-d '{"busAddress": "0x9A", "ch1": 20, "ch2": 20}'
```

5.5 单独改变一台设备的 CH1 或 CH2：必须保留另一通道原值

现有网关未提供单独 **set-channel** HTTP 路径。/led/set 每次会同时写入两个通道。例如想把 0x9B 的 CH1 调到 100、CH2 保持原值，应先读 0x9B，假设确认其 CH2 当前是 20 后，才发送：

```
curl -X POST "http://127.0.0.1:8090/api/v1/led/set" \
-H "Content-Type: application/json" \
-d '{"busAddress": "0x9B", "ch1": 100, "ch2": 20}'
```

上例中的 20 只是示例，**必须用实时读回的 CH2 数值替换**；否则会意外改变该通道。

5.6 关闭全部四台 LED 的所有通道（广播全灭）

该命令会改变全部 8 路灯光，请确认现场允许。

```
curl -i -X POST "http://127.0.0.1:8090/api/v1/led/broadcast/set" \
-H "Content-Type: application/json" \
-d '{"ch1": 0, "ch2": 0}'
```

期望响应结构：

```
{"ok": true, "scope": "BROADCAST", "ch1": 0, "ch2": 0, "note": "broadcast may have no per-device ack"}
```

必须再逐台确认：

```
curl "http://127.0.0.1:8090/api/v1/led/read-all"
```

四台设备均 ok:true 且 ch1=0, ch2=0 时，可认为八个通道的控制寄存器均确认置 0；这仍不是独立光学传感器测量。

5.7 全部四台设备两通道全亮（广播）

警告：可能让全部灯瞬间达到最大协议设置值，仅在现场允许的情况下执行。

```
curl -i -X POST "http://127.0.0.1:8090/api/v1/led/broadcast/set" \
-H "Content-Type: application/json" \
-d '{"ch1":255,"ch2":255}'
```

5.8 广播设置统一亮度 / 只亮同一通道

所有设备 CH1=100、CH2=50:

```
curl -X POST "http://127.0.0.1:8090/api/v1/led/broadcast/set" \
-H "Content-Type: application/json" \
-d '{"ch1":100,"ch2":50}'
```

所有设备仅 CH1 开到 255, CH2 关闭:

```
curl -X POST "http://127.0.0.1:8090/api/v1/led/broadcast/set" \
-H "Content-Type: application/json" \
-d '{"ch1":255,"ch2":0}'
```

所有设备仅 CH2 开到 255, CH1 关闭:

```
curl -X POST "http://127.0.0.1:8090/api/v1/led/broadcast/set" \
-H "Content-Type: application/json" \
-d '{"ch1":0,"ch2":255}'
```

广播写入后, 用下面这条指令统一校验:

```
curl "http://127.0.0.1:8090/api/v1/led/read-all"
```

广播与单台写入的关键区别: 单台 set 等待设备返回 Modbus 写入确认帧; 广播地址 0x00 通常没有逐台应答, HTTP 的 ok:true 只代表串口广播帧已发送, 不保证各设备全部完成控制。未读回成功时, 应按设备逐台排查。

6. 在当前 Conda base 环境启动 / 停止网关

6.1 首次检查

```
cd ~/project/led_linux_gateway
which python
python --version
python -c "import serial, fastapi, uvicorn; print('依赖正常')"
ls -l /dev/ttyUSB0
groups
```

当前串口文件通常属于 root:dialout; 若 groups 没有 dialout, 执行:

```
sudo usermod -aG dialout hzauiot
newgrp dialout
```

newgrp 作用于新 Shell; 永久生效建议重新登录系统。不要长期把串口权限设置为 777。

6.2 本机手动启动 (当前使用的方式)

在 Conda base 环境下执行:

```
cd ~/project/led_linux_gateway
LED_SERIAL_PORT=/dev/ttyUSB0 \
LED_HTTP_HOST=127.0.0.1 \
LED_HTTP_PORT=8090 \
python server.py
```

这会在前台运行, 终端不能关闭。另开终端执行 curl 即可调用接口。正常会显示 Uvicorn running on http://127.0.0.1:8090 之类日志。

6.3 关闭手动服务

在运行 python server.py 的终端按 Ctrl+C。如果不清楚程序是否在运行, 可先检查:

```
sudo ss -lntp | grep ':8090'
```

不要同时启动多个网关进程, 也不要与原 PyQt 串口程序同时占用 /dev/ttyUSB0。

6.4 如需从其他电脑访问

先确认工控机 IP：

```
hostname -I
```

设置强密钥后，通过以下命令启动（只是示例，建议正式环境把密钥存到权限受限的配置文件或 systemd 的 EnvironmentFile 中）：

```
cd ~/project/led_linux_gateway
read -rs -p "输入新的 LED API Key: " LED_KEY; echo
LED_SERIAL_PORT=/dev/ttyUSB0 \
LED_HTTP_HOST=0.0.0.0 \
LED_HTTP_PORT=8090 \
LED_API_KEY="$LED_KEY" \
python server.py
```

注意：0.0.0.0 是服务监听所有网卡，不是客户端访问地址。平台访问时必须使用工控机真实局域网 IP，并携带相同 API Key。

在平台端（或局域网另一台电脑）测试：

```
IP=<工控机真实 IP>
read -rs -p "输入 LED API Key: " LED_KEY; echo
curl -i -H "X-API-Key: $LED_KEY" "http://$IP:8090/api/v1/led/health"
curl -i -H "X-API-Key: $LED_KEY" "http://$IP:8090/api/v1/led/read-all"
```

端口建议仅允许平台服务器 IP 在可信局域网内访问。HTTP 未加密，不建议直接暴露到公网。

6.5 后台运行与开机自启说明

项目自带的 led-gateway.service 针对 /opt/led-gateway/.venv/bin/python 和用户 ledgateway 编写，不能在当前 ~/project/led_linux_gateway + Conda base 的环境中不做修改就照搬。

如果需要长期运行，建议优先按照工程的 README.md 将服务部署到 /opt/led-gateway 和独立虚拟环境，再安装对应 systemd 服务。若坚持使用 Conda base，应先确认 which python 的绝对路径并修改服务中的 WorkingDirectory、ExecStart、User、EnvironmentFile 等参数；ProtectHome=true 也可能阻止服务读取 ~/project。在验证前，不应把未改动的原 service 直接启用。

7. 接口入参、返回与错误码

7.1 参数规则

参数	类型 / 范围	说明
busAddress	JSON 字符串，0x9A~0x9D	指定设备地址；GET 放 URL 参数中
ch1	JSON 整数，0~255	第一通道光强
ch2	JSON 整数，0~255	第二通道光强
X-API-Key	HTTP 请求头	已配置 LED_API_KEY 时必填
rackKey	不需要传给网关	由平台侧维护（如 led-rack-01）

7.2 常见 HTTP 状态码

状态码	含义	排查重点
200	请求成功；请结合 JSON 判断细节	广播需要后续回读
401	API Key 缺失或错误	X-API-Key 与 LED_API_KEY 是否一致
422	请求参数不合法	地址格式、缺少通道、值不是整数或超范围
502	单台设备 Modbus 通信/协议异常	接线、地址、响应超时、CRC、功能码
503	单台接口无法打开/使用串口	串口路径、权限、串口被占用

错误体通常类似 {"detail": "错误原因"}。特例：read-all 为了保留其他设备的结果，会在某台设备错误时返回 HTTP 200，并在该设备的对象内写 ok:false、error、ch1:null、ch2:null；因此要逐个检查 devices[].ok，不能只检查 HTTP 状态码。

7.3 常见参数错误示例（不要直接执行写入）

```
{ "busAddress": "0x9E", "ch1": 20, "ch2": 20 }
```

错误：设备地址不在允许范围内。

```
{ "busAddress": "0x9A", "ch1": "20", "ch2": 20 }
```

错误：ch1 为字符串而非整数。

```
{ "busAddress": "0x9A", "ch1": 300, "ch2": 20 }
```

错误：ch1 超出 0~255。

8. 日常故障排查命令

现象	推荐诊断指令	解释
找不到串口	ls -l /dev/ttyUSB*	设备节点是否存在
CH340 识别异常	lsusb、lsusb -t	查看 USB 设备与驱动绑定
驱动日志	sudo dmesg grep -Ei 'ch34 ttyUSB brltty' tail -n 50	查看是否被 brltty 等服务干扰
无串口权限	groups、ls -l /dev/ttyUSB0	用户是否在 dialout 组
HTTP 8090 无响应	sudo ss -lntp grep ':8090'	服务是否正在监听
Python 依赖异常	python -c "import serial, fastapi, uvicorn"	当前环境是否有依赖
单台通信超时	调用 /led/read?busAddress=...	先排查对应设备地址及 A/B 接线
广播后设备状态不一致	调用 /led/read-all	逐台确认，广播本身无逐台应答

注：上表中 \| 仅为表格中避免竖线分列的转义写法；实际执行如下命令，不带反斜杠转义：

```
sudo dmesg | grep -Ei 'ch34|ttyUSB|brltty' | tail -n 50
sudo ss -lntp | grep ':8090'
```

如果端口 8090 已被占用，请先辨认监听进程，不要随意结束其他服务。若串口曾因 brltty 冲突解绑（本工控机曾出现），需要先消除服务冲突、重新插拔转换器并确认 /dev/ttyUSB0 恢复，然后再启动网关。

9. 操作安全与平台对接注意事项

读取优先、写入后回读。先 read-all 建立基线，再执行改变光强的命令，最后再 read-all 复核。

单台设置也会一次写两个通道。希望“只改 CH1”时必须读出并保留原 CH2 值，反之亦然。

广播不等于逐台确认。发生通信异常时，单台 set 与广播返回值有不同含义。

不要自动无限重试写操作。在是否已写入不确定时先读取确认，以免造成不期望的连续控制。

谨慎处理离线结果。read-all 中某台 ok:false 时平台应标记离线，保留最近一次确认值，不要把 null 当成 0。

网关只负责硬件接口。平台侧的 /plant/api/led/...、plant_led_state、plant_led_command_log 不由本网关直接创建或维护。

限制网络访问。生产环境使用有效的 API Key、受控局域网、防火墙白名单；/docs 页面默认不做 API Key 验证。

设备物理特性以实机为准。寄存器写入成功以及回读一致，仅能确认控制器逻辑状态；如需判断真实光输出，应通过现场观察或光学传感器测量。

10. 现场操作速查：最常用的五条指令

① 读单台 0x9A

```
curl "http://127.0.0.1:8090/api/v1/led/read?busAddress=0x9A"
```

② 读全部四台

```
curl "http://127.0.0.1:8090/api/v1/led/read-all"
```

③ 设置 0x9A 的 CH1=30、CH2=20

```
curl -X POST "http://127.0.0.1:8090/api/v1/led/set" -H "Content-Type: application/json" -d '{"busAddress":"0x9A","ch1":30,"ch2":20}'
```

④ 所有灯关闭

```
curl -X POST "http://127.0.0.1:8090/api/v1/led/broadcast/set" -H "Content-Type: application/json" -d '{"ch1":0,"ch2":0}'
```

⑤ 所有灯最大设置值（谨慎）

```
curl -X POST "http://127.0.0.1:8090/api/v1/led/broadcast/set" -H "Content-Type: application/json" -d '{"ch1":255,"ch2":255}'
```

标准操作顺序： read-all → 写入/广播 → read-all 核实。以上写入命令会直接作用于实际灯光。

文档范围： 仅对应当前 Linux 工控机 LED 网关的 HTTP 接口②，不包含平台后端数据库接口①，不包含二期的组控制、定时配方及 EEPROM 保存功能。部署步骤与原始工程实际配置一致；现场测试结果仅在第 1 节按对话记录明确列出。